Aula 09 - 3FN e Revisão Geral da Modelagem

Descrição: Aula focada na identificação e eliminação de dependências transitivas para alcançar a Terceira Forma Normal (3FN). Realização de atividade prática de elaboração dos documentos: Ficha de Levantamento de Requisitos, criação do Modelo de Entidade-Relacionamento (MER) seguida da validação final do Diagrama Entidade-Relacionamento (DER) e organização da documentação técnica para a Situação de Aprendizagem.

Cenário Real: Pensem no banco de dados de um aplicativo de delivery que vocês usam todo fim de semana. Imagine que, na tabela de "Pedidos", temos as colunas: ID_Pedido, Nome_Lanche, Bairro_Entrega e Taxa_Entrega. A Taxa_Entrega é definida pelo Bairro_Entrega, e não pelo ID_Pedido. Se o aplicativo decidir aumentar a taxa de entrega do bairro "Centro" de R$ 5,00 para R$ 7,00, o banco de dados precisará varrer e atualizar milhares de pedidos antigos só para manter a consistência, ou pior, teremos dados divergentes. Esse é o caos causado por uma dependência transitiva.

Pergunta: Como podemos reestruturar os dados desse aplicativo para que a alteração da taxa de um bairro seja feita em um único lugar, refletindo instantaneamente para todos os novos pedidos sem corromper o histórico?

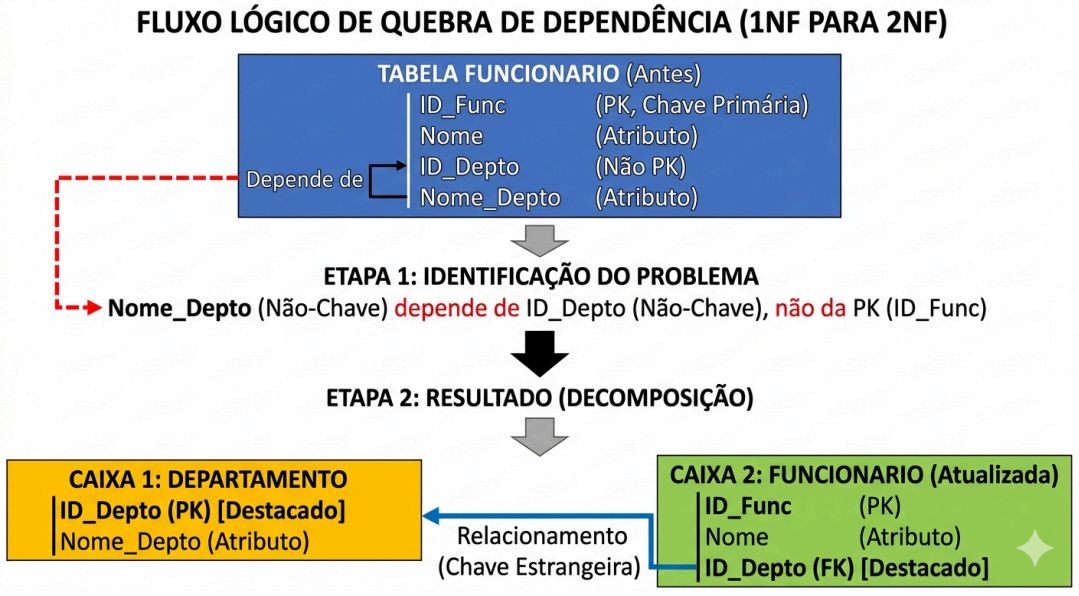
Terceira Forma Normal (3FN) e Dependências Transitivas

A. Conceito: Para que uma tabela esteja na Terceira Forma Normal (3FN), ela precisa obrigatoriamente já estar na **1FN** e na **2FN**. A regra de ouro da 3FN é clara: **toda coluna que não é chave primária deve depender APENAS da chave primária**. Se um atributo não-chave depende de outro atributo que também não é chave, temos uma **dependência transitiva**. No nosso dia a dia, isso significa que estamos misturando fatos sobre entidades independentes na mesma tabela, o que gera redundância e anomalias de atualização.

Para resolver isso, nós pegamos o **atributo independente** (como o Bairro e sua Taxa) e o transformamos em **uma nova tabela**, criando um relacionamento via **Chave Estrangeira** (FK). É aqui que entra o pensamento crítico: questionar sempre a origem do dado.

Podemos também utilizar ferramentas de IA Generativa para escanear grandes dicionários de dados e sugerir tabelas candidatas à normalização, mas a validação do modelo de negócios sempre será responsabilidade do desenvolvedor.

B. Visualizando:



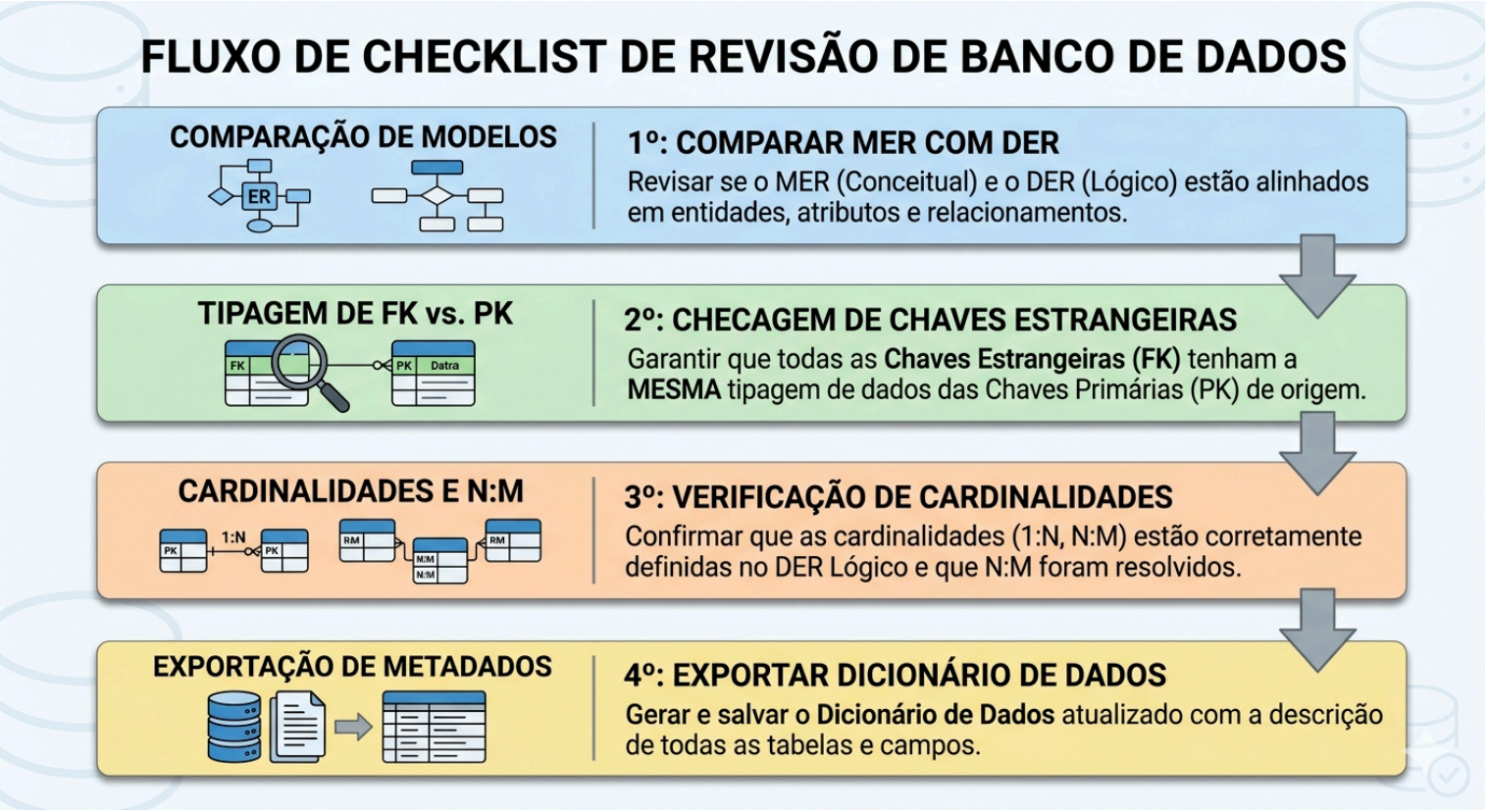
C. Perguntas:
1. O que caracteriza uma dependência transitiva?
2. Por que a 3FN exige que a tabela já esteja na 2FN?
3. Cite um problema prático causado por manter atributos com dependência transitiva na mesma tabela.
4. No exemplo do código, o que aconteceria se tentássemos excluir um `Bairro` que já possui pedidos vinculados na tabela `Pedidos_Correto`?
5. Como uma FK (Chave Estrangeira) ajuda a manter a 3FN?

Revisão do DER e Documentação Técnica

A. Conceito: Após aplicar todas as regras de normalização (**1FN, 2FN e 3FN**), nosso modelo lógico sofreu diversas alterações — tabelas foram divididas e novos relacionamentos nasceram. Agora, nós precisamos revisar o Diagrama Entidade-Relacionamento (**DER**) para garantir que ele reflita exatamente o banco de dados que será implementado. Isso garante a rastreabilidade do projeto.

A preparação para a avaliação exige documentação impecável. Precisamos consolidar o Dicionário de Dados, descrevendo os tipos, tamanhos e restrições de cada coluna. Um profissional de desenvolvimento não entrega **apenas código**; ele entrega **soluções bem documentadas**. A postura ética e o profissionalismo exigem clareza técnica, para que qualquer outro desenvolvedor da equipe (ou o professor, na Situação de Aprendizagem) consiga compreender e dar manutenção na estrutura criada.

B. Visualizando:



FAQ - Dúvidas Reais

1. Pergunta: Preciso normalizar o banco de dados até a 5FN em todos os projetos?

Resposta: Na prática da indústria, normalizar até a 3FN (ou Forma Normal de Boyce-Codd - FNBC) já resolve 95% das anomalias. Formas normais superiores são para casos muito específicos e podem prejudicar a performance.

2. Pergunta: E se a 3FN deixar meu banco lento por causa de muitos JOINs?
Resposta: Esse é um trade-off clássico. Em bancos de leitura massiva (como Data Warehouses), fazemos a *desnormalização controlada*. Mas em sistemas transacionais (OLTP) do dia a dia, a 3FN é a regra.

3. Pergunta: Como sei se um campo é realmente independente para virar uma tabela?
Resposta: Pergunte-se: "Se eu deletar o registro principal, esse dado deveria continuar existindo no sistema?". Se sim (ex: a categoria de um produto que foi apagado), ele deve ser uma tabela à parte.

4. Pergunta: Posso usar IA para gerar meu dicionário de dados?
Resposta: Pode e deve para ganhar tempo na estruturação inicial (Markdown ou planilhas), mas você deve revisar cada tipo de dado, pois a IA não conhece as regras de negócio intrínsecas da sua empresa.

5. Pergunta: Esqueci de colocar a FK no DER, posso criar direto no SQL?
Resposta: Tecnicamente sim, mas você quebra a rastreabilidade (documentação vs. código). A boa prática exige que você atualize o DER primeiro.

ATIVIDADE DE CONSTRUÇÃO - DOCUMENTAÇÃO DA SA

Fluxo de Trabalho

1. Levantamento de Requisitos: Converse com o cliente/usuário para entender a regra de negócio. Documentos a serem apresentados:

- Canvas;
- Ficha de Levantamento de Requisitos:
    - Funcionais,
    - Não Funcionais,
    - Regras de Negócio.

---

2. MER (Modelo Conceitual): Identifique entidades e relacionamentos (**Documento de Texto**).

Criar um **Modelo de Entidade-Relacionamento (MER)** é como desenhar a planta baixa de um prédio antes de começar a assentar os tijolos (o banco de dados). É uma representação conceitual que descreve os dados, as regras de negócio e como as informações se conectam.

Aqui está o passo a passo detalhado para construir um MER:

2.1. Identifique as Entidades (Os Substantivos)
As entidades são os objetos do mundo real sobre os quais você deseja armazenar informações. Geralmente são **substantivos**.

- **Exemplo:** Em um sistema de e-commerce, as entidades seriam `CLIENTE`, `PRODUTO`, `PEDIDO`.
- **Representação:** No diagrama (DER), são desenhadas dentro de **retângulos**.

2.2. Defina os Atributos (As Características)

Cada entidade possui características que a descrevem.

- **Exemplo:** A entidade CLIENTE tem os atributos: Nome, CPF, E-mail, Data de Nascimento.
- **Atributo Chave (Chave Primária):** É o identificador único daquela entidade (ex: ID_Cliente ou CPF). Ele garante que nenhum registro seja igual a outro.
- **Representação:** Geralmente desenhados como **elipses** ligadas à entidade ou listados dentro do retângulo.

2.3. Estabeleça os Relacionamentos (Os Verbos)

Como as entidades interagem entre si? O relacionamento é a associação entre elas, geralmente um **verbo**.

- **Exemplo:** CLIENTE **faz** PEDIDO; PEDIDO **contém** PRODUTO.
- **Representação:** No diagrama, são representados por um **losango** entre as entidades.

2.4. Defina a Cardinalidade

Este é o passo onde muitos se confundem, mas é crucial. A cardinalidade define o "número de ocorrências" no relacionamento. Os tipos mais comuns são:

- **1:1 (Um para Um):** Um usuário possui apenas um currículo.
- **1:N (Um para Muitos):** Um cliente pode fazer vários pedidos, mas cada pedido pertence a apenas um cliente.
- **N:N (Muitos para Muitos):** Um aluno pode se matricular em várias disciplinas, e uma disciplina pode ter vários alunos.
  Dica técnica: No modelo lógico, relacionamentos **N:N** geram uma nova tabela (tabela associativa).

2.5. Exemplo Prático: Sistema de Biblioteca

Para fixar o conhecimento, vamos montar a estrutura lógica:

1. **Entidades:** AUTOR, LIVRO.
2. **Atributos:** * AUTOR: ID_Autor (PK), Nome, Nacionalidade.
   - LIVRO: ISBN (PK), Título, Ano_Publicação.
3. **Relacionamento:** AUTOR **escreve** LIVRO.
4. **Cardinalidade:** Um autor pode escrever **N** livros. Um livro pode ser escrito por **N** autores (coautoria). Portanto, é um relacionamento **N:N**.

---

3. DER (Diagrama lógico): Desenhe o modelo usando as ferramentas visuais, definindo chaves primárias e estrangeiras. (Utilize a ferramenta de sua preferência)

Transformar o **MER (Modelo de Entidade-Relacionamento)** em um **DER (Diagrama de Entidade-Relacionamento)** é o passo em que passamos da teoria e do conceito para uma estrutura mais próxima do banco de dados real. No mercado, essa transição é conhecida como passar do **Modelo Conceitual** para o **Modelo Lógico**.

Aqui está o passo a passo detalhado para realizar essa conversão:

3.1. Transformar Entidades em Tabelas

Esta é a parte mais simples: cada retângulo (entidade) do seu MER vira uma tabela no seu DER.

- **Ação:** Desenhe a estrutura da tabela com o nome da entidade no topo.

3.2. Definir as Chaves Primárias (Primary Keys - PK)

Todo registro em uma tabela precisa de um identificador exclusivo.

- **Ação:** Escolha o atributo identificador do MER (como ID_Cliente ou CPF) e marque-o como **PK**. Se não houver um identificador natural, crie um campo "ID" auto-incremento.

3.3. Mapear Atributos para Colunas e Tipos de Dados

No MER, você só lista o nome da característica. No DER, você precisa dizer o que ela é.

- **Ação:** Para cada atributo, defina o tipo de dado:
  - `Nome` -> `VARCHAR(100)` (Texto)
  - `Idade` -> `INT` (Inteiro)
  - `Preço` -> `DECIMAL(10,2)` (Moeda)
  - `Data` -> `DATE` ou `DATETIME`.

3.4. Converter Relacionamentos (O Coração do DER)

Aqui é onde aplicamos as regras de **Chave Estrangeira (Foreign Key - FK)** com base na cardinalidade:

A) Relacionamento 1:N (Um para Muitos)

Esta é a regra mais comum. A Chave Primária do lado "1" deve "viajar" para a tabela do lado "N" para se tornar uma Chave Estrangeira.

- **Exemplo:** Um `CLIENTE` (1) faz vários `PEDIDOS` (N).
- **No DER:** A tabela `PEDIDO` ganhará uma nova coluna chamada `ID_Cliente (FK)`, que se conecta ao `ID_Cliente (PK)` da tabela Cliente.

B) Relacionamento N:N (Muitos para Muitos)

No DER, bancos de dados relacionais não conseguem ler um "N:N" direto. Você deve criar uma **Tabela Associativa** (ou Tabela de Ligação).

- **Exemplo:** `LIVRO` (N) --- `AUTOR` (N).
- **No DER:** Crie uma terceira tabela chamada `AUTOR_LIVRO`. Ela conterá, no mínimo, duas colunas: `ID_Autor (FK)` e `ID_Livro (FK)`. Juntas, elas formam a chave primária composta dessa nova tabela.

C) Relacionamento 1:1 (Um para Um)

A Chave Primária de uma tabela vai para a outra como Chave Estrangeira. Geralmente escolhe-se a tabela que "depende" da outra para existir.

3.5. Aplicar Regras de Integridade e Nulidade

Agora você define o que é obrigatório ou não:

- **NOT NULL:** O campo deve ser preenchido obrigatoriamente (ex: E-mail de um usuário).
- **NULL:** O campo pode ficar vazio (ex: Complemento de endereço).
- **UNIQUE:** O valor não pode se repetir (ex: CPF, mesmo que não seja a PK).

3.6. Desenhar as Linhas de Conexão (Notação)

No DER, usamos linhas para conectar as tabelas. As notações mais famosas são:

- **Pé de Galinha (Crow's Foot):** Usa símbolos nas extremidades das linhas (um traço para "1" e três ramificações para "Muitos").
- **UML:** Usa números (1..1, 1..*) nas pontas das conexões.

Resumo Visual da Conversão:

No MER (Conceitual)	No DER (Lógico)
Entidade	Tabela
Atributo	Coluna
Identificador	Chave Primária (PK)
Relacionamento 1:N	Chave Estrangeira (FK) no lado N

Relacionamento N:N	Nova Tabela Associativa

Fechamento

Conexão com a Situação de Aprendizagem:
O que nós construímos e revisamos hoje é exatamente um dos critérios fundamentais da prova prática de vocês: "Aplicação de normalização até a 3ª forma normal". Se o modelo de vocês chegar na fase de codificação com dependências transitivas, os relatórios futuros darão resultados errados, comprometendo a entrega final. O dicionário de dados organizado hoje será anexado como o principal documento técnico de arquitetura na apresentação final.

Próxima Aula:
Na próxima aula realizaremos uma avaliação objetiva para validar o conhecimento adquirido até este momento. Nas próximas etapas, sairemos da prancheta de modelagem (DDL) e entraremos forte na manipulação dos dados (DML). Faremos as inserções (`INSERT`) com base no nosso modelo 3FN e começaremos a testar se nossos relacionamentos suportam consultas reais usando `SELECT` e primeiros cruzamentos de dados.